Combinational Circuits - II

109

109

Agenda

- Encoder & Decoder
- Mux & DeMux
- Tristate Buffer

110

110

Encoders

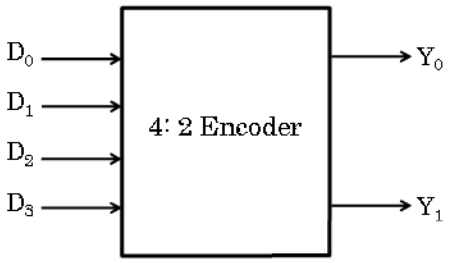
Encoders

- Converts Human understandable into machine understandable codes
- Assigns a binary code to an active input line.
- One hot to binary converter,
- Produces n no. of outputs when there is 2^n no. of inputs,
- At most only one of the inputs will ever be high, the binary code of this 'hot' line is produced

111

111

4:2 Encoder



D3	D2	D1	D0	Y1	Y0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Outputs the binary value of the active input

112

112

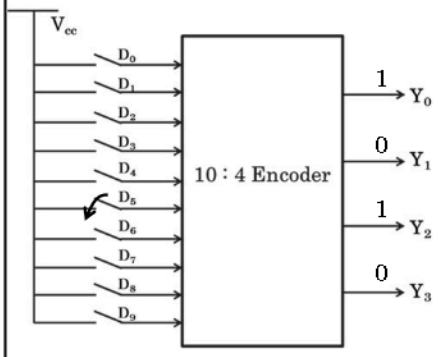
Decimal to Binary Encoder

Specifications

- Take 10 keys as Decimal inputs & produce 4 bit Binary Output based on the input key

What if more than one input is active?
What if no inputs are active?

Block Diagram, Truth Table



D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	Y3	Y2	Y1	Y0
0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	1	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	0	1	0	0	0	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	0	0	0	0	1	1	1
0	1	0	0	0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0	0	0	1	0	0	1
Remaining Combinations										X	X	X	X

113

113

Priority Encoder

Priority Encoders

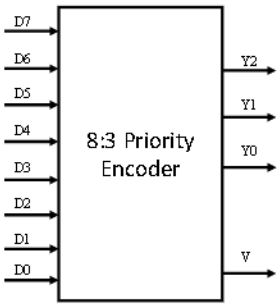
- Assign priorities to the inputs when more than one input are asserted simultaneously,
- Highest priority input is taken into account to produce the binary output.
- At least any one of the input line should be active, to make the encoder valid. A 'valid' indicator, is included to indicate the same.

114

114

8:3 Priority Encoder

Priority to higher order inputs.



D7	D6	D5	D4	D3	D2	D1	D0	Y2	Y1	Y0	Valid
0	0	0	0	0	0	0	0	X	X	X	0
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	1	X	0	0	1	1
0	0	0	0	0	1	X	X	0	1	0	1
0	0	0	0	1	X	X	X	0	1	1	1
0	0	0	1	X	X	X	X	1	0	0	1
0	0	1	X	X	X	X	X	1	0	1	1
0	1	X	X	X	X	X	X	1	1	0	1
1	X	X	X	X	X	X	X	1	1	1	1

115

115

ASCII Encoder

- ASCII stands for American Standard Code for Information Interchange

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	EF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2		!	--	#	$	%	&	.	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	:	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	/	]	^	_
6	.	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	l	}	~	DEL

- The code uses 7 bits to encode 128 unique characters

116

116

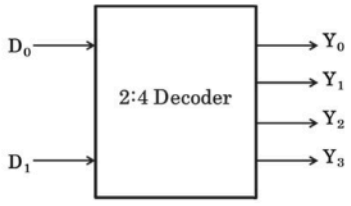
Decoders

- Covert Machine understandable into Human understandable codes
- Convert binary information from n input signals to 2^n unique output signals
- used in a wide variety of applications, including data demultiplexing, seven segment displays, and memory address decoding.

117

117

2:4 Decoder



D1	D0	Y3	Y2	Y1	Y0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

$Y0 = \overline{D1}.\overline{D0}$ $Y1 = \overline{D1}.D0$ $Y2 = D1.\overline{D0}$ $Y3 = D1.D0$

Possible minterms of D1 and D0

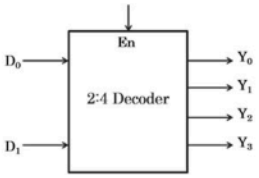
With N to 2^N Decoder, N Variable function can be implemented

118

118

Decoders with Active High & Low O/P with Enable inputs.

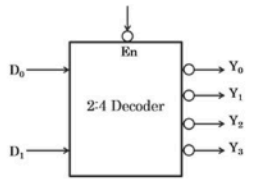
Decoder with Active HIGH Output & active HIGH Enable



En	D1	D0	Y3	Y2	Y1	Y0
0	X	X	X	X	X	X
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

Generates all possible minterms

Decoder with Active LOW Output & Active LOW Enable



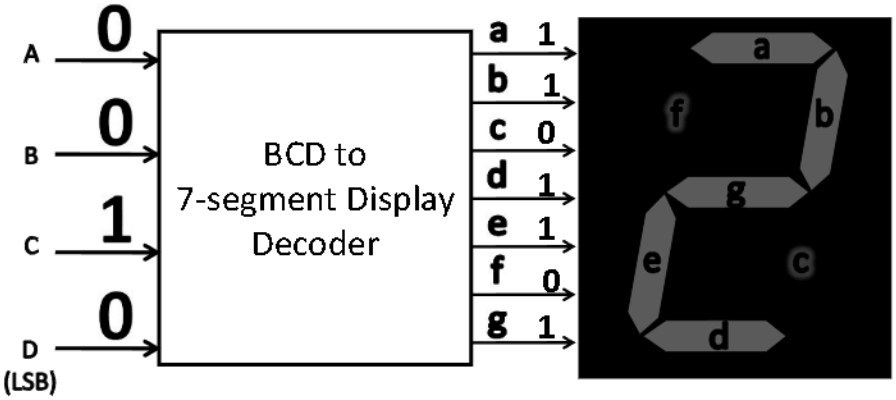
En	D1	D0	Y3	Y2	Y1	Y0
1	X	X	X	X	X	X
0	0	0	1	1	1	0
0	0	1	1	1	0	1
0	1	0	1	0	1	1
0	1	1	0	1	1	1

Generates all Possible maxterms

119

119

BCD-to-Seven Segment Decoder



120

120

Circuit Design using Decoders

- Any combinational circuit with n inputs and m outputs can be implemented with an n-to-2^n decoder with m OR gates because decoder with active HIGH output produces all possible minterms.
- Similarly, Any combinational circuit with n inputs and m outputs can be implemented with an n-to-2^n decoder with m AND gates because decoder with active LOW output produces all possible maxterms.

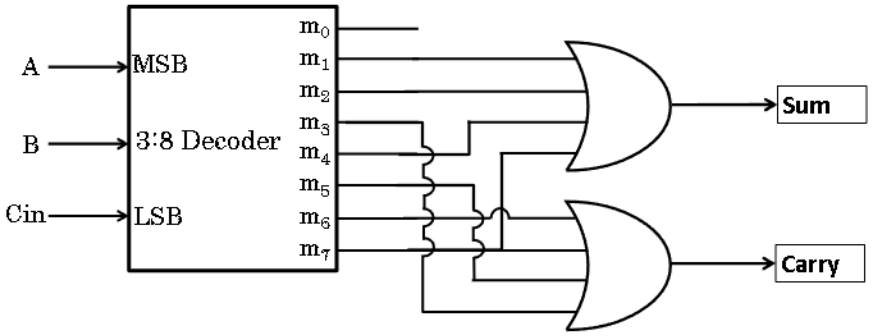
121

121

Fulladder using Decoders

Example : **Realize a full adder using a 3 to 8 decoder using Active HIGH output decoder.**

$Sum = \sum m(1,2,4,7)$ $Cout = \sum m(3,5,6,7)$



122

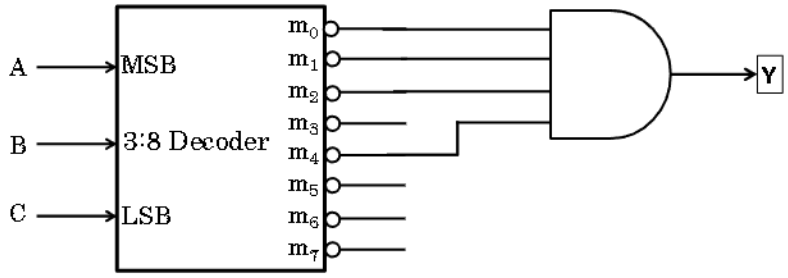
122

Boolean Functions using Decoders

Example : **Implement the following function using an active LOW output decoder.**

$$Y = \prod M(0,1,2,4)$$

For implementing any function using active low output decoder, the function should be in canonical POS form



123

123

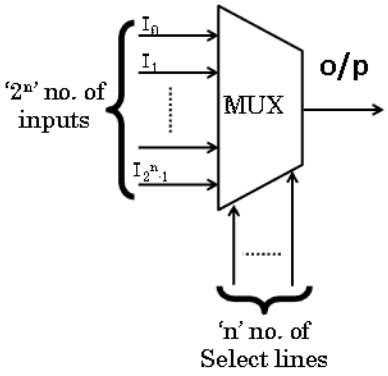
MUX & DeMUX

124

124

Multiplexer (MUX)

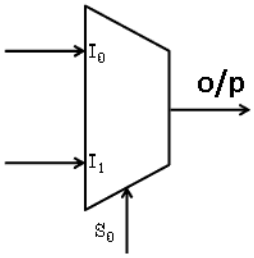
Logic Symbol & Functionality



- Single output line
- One of the data input is routed to the output based on the select signals.

125

2:1 MUX



Truth Table

S_0	I_1	I_0	O/P	
0	0	0	0	O/P same as I_0
0	0	1	1	
0	1	0	0	
0	1	1	1	
1	0	0	0	O/P same as I_1
1	0	1	0	
1	1	0	1	
1	1	1	1	

Functional Table

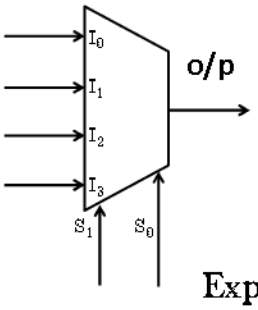
S_0	O/P
0	I_0
1	I_1

$Y = S_0'I_0 + S_0I_1$

$$output\ (Y) = \sum m(1,3,6,7)$$

126

4:1 MUX



Functional Table

S_1	S_0	O/P
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Expression for output

$$Y = \overline{S_1}\,\overline{S_0} I_0 + \overline{S_1} S_0 I_1 + S_1 \overline{S_0} I_2 + S_1 S_0 I_3$$

127

127

MUX – Universal Logic

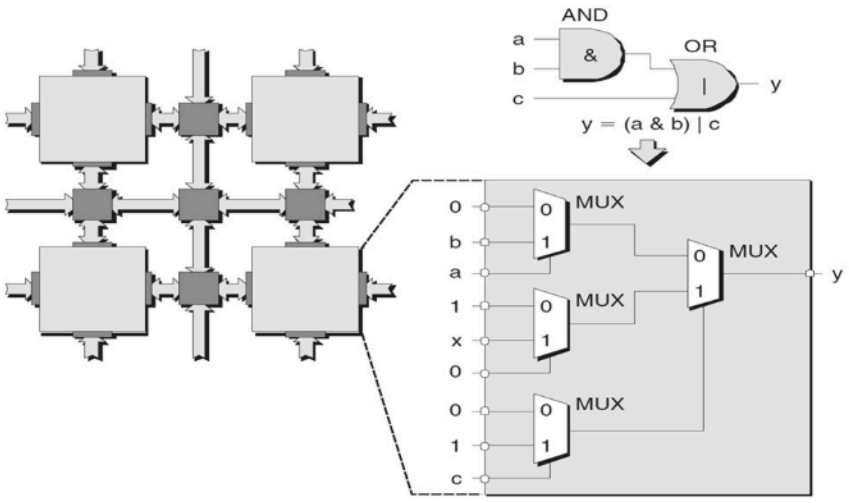
- Similar to NAND & NOR, MUX can also be used to realize any Boolean Function.
- MUX – Universal Logic Block

- Functions are in Canonical Form - Use truth tables
- Not in Canonical Form – Use Shannon's Expression.

128

128

MUX – Universal Logic



129

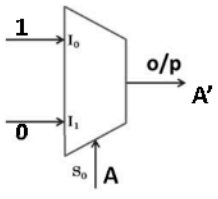
129

Implementing Logic Gates using MUX

MUX as NOT gate

- If input is A, then output should be complement of A.

A	O/P (Y)	
0	1	I_0
1	0	I_1



Input Signal as Select Signal

Outputs as Data Inputs

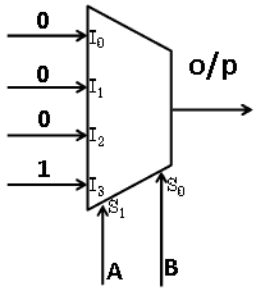
130

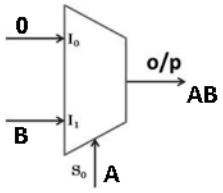
130

Implementing Logic Gates using MUX

MUX as AND gate

A	B	O/P (Y)	
0	0	0	$I_0 = 0$
0	1	0	
1	0	0	$I_1 = B$
1	1	1	



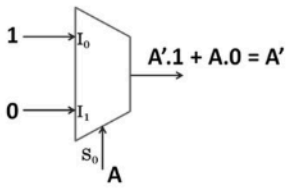


131

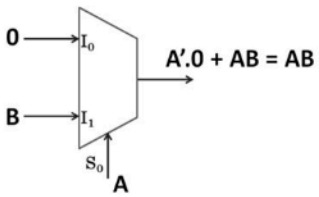
131

Implementing Logic Gates using MUX

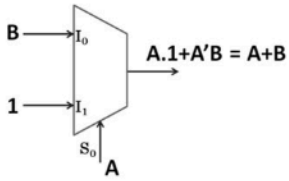
NOT gate



AND gate



OR gate

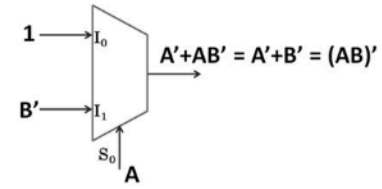


132

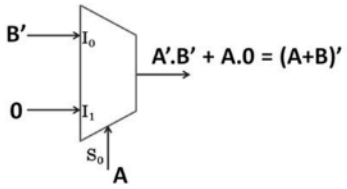
132

Implementing Logic Gates using MUX

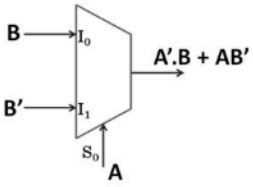
NAND gate



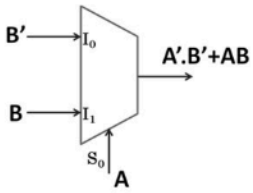
NOR gate



EXOR gate



EXNOR gate



133

133

Circuit Design Using MUX

Implement the following function using 16:1 MUX

$$Y = f(A,B,C,D) = \sum m(1,2,3,6,7,9,13,14)$$

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

- Since, the no. of literals in the expression & the no. of select lines is 4, all the input literals can be used as select lines.

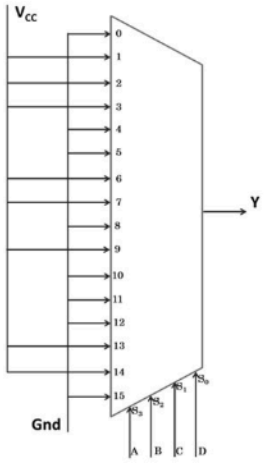
134

134

Function Implementation Using MUX

$$Y = f(A,B,C,D) = \sum m(1,2,3,6,7,9,13,14)$$

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0



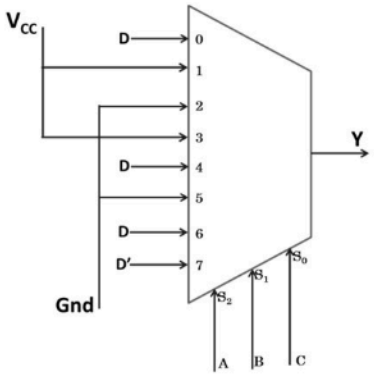
135

135

Implement using 8:1 MUX

$$Y = f(A,B,C,D) = \sum m(1,2,3,6,7,9,13,14)$$

A	B	C	D	Y	
0	0	0	0	0	$I_0 = D$
0	0	0	1	1	
0	0	1	0	1	$I_1 = 1$
0	0	1	1	1	
0	1	0	0	0	$I_2 = 0$
0	1	0	1	0	
0	1	1	0	1	$I_3 = 1$
0	1	1	1	1	
1	0	0	0	0	$I_4 = D$
1	0	0	1	1	
1	0	1	0	0	$I_5 = 0$
1	0	1	1	0	
1	1	0	0	0	$I_6 = D$
1	1	0	1	1	
1	1	1	0	1	$I_7 = D'$
1	1	1	1	0	



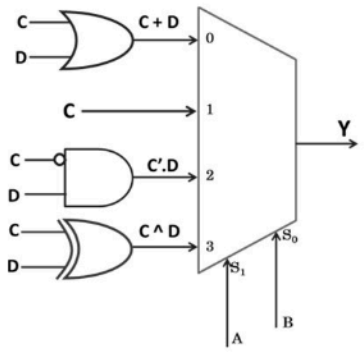
136

136

Implement Using 4: 1 MUX

$$Y = f(A,B,C,D) = \sum m(1,2,3,6,7,9,13,14)$$

A	B	C	D	Y	
0	0	0	0	0	
0	0	0	1	1	$I_0 = C + D$
0	0	1	0	1	
0	0	1	1	1	
0	1	0	0	0	
0	1	0	1	0	$I_1 = C$
0	1	1	0	1	
0	1	1	1	1	
1	0	0	0	0	
1	0	0	1	1	$I_2 = C'D$
1	0	1	0	0	
1	0	1	1	0	
1	1	0	0	0	
1	1	0	1	1	$I_3 = C \wedge D$
1	1	1	0	1	
1	1	1	1	0	

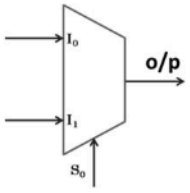


137

137

MUX - Shannon's Expression

2: 1 MUX



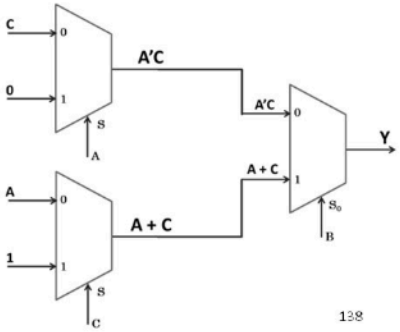
Expression for the output

$$Y = \overline{S_0} I_0 + + S_1 I_1$$

Known as Shannon's Expression for 2:1 MUX

$$Y = BC + AB\overline{C} + \overline{A}\,\overline{B}C$$

1. Bring this expression into the form of Shannon's Expression.
2. Then draw using 2:1 MUX
3. You can learn this in detail during Advance Digital course



138

138

Hierarchical Design

- Several issues arise when designing large multiplexers(as 2-level circuits).
  - Number of logic gates gets prohibitively large
  - Number of inputs to each logic gate (i.e. fan-in) gets prohibitively large
- Instead, design hierarchically
  - Use smaller elements as building blocks
  - Interconnect building blocks in a multi-tier structure

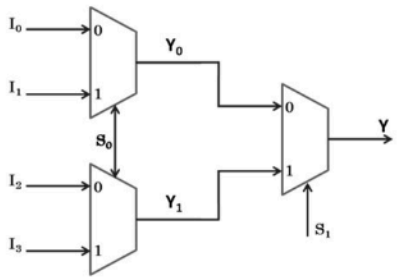
139

139

4:1 MUX using 2:1 MUX

Truth Table

S_1	S_0	O/P	
0	0	I_0	Y_0
0	1	I_1	
1	0	I_2	Y_1
1	1	I_3	



S_1	O/P
0	Y_0
1	Y_1

1. I0 or I1 can be selected by using a 2:1 MUX using S0 as the select line signal.
2. I2 or I3 can be selected by using another 2:1 MUX using S0 as the select line signal.
3. Y0 or Y1 can be selected by using another 2:1 MUX using S1 as the select line signal

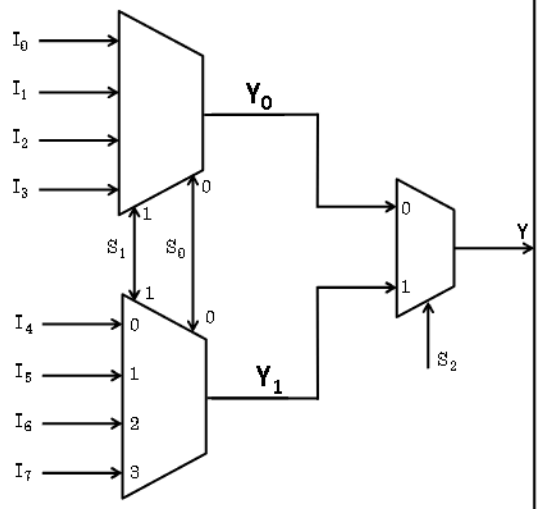
140

140

8:1 MUX using 4:1 & 2:1 MUX

Truth Table

S_2	S_1	S_0	O/P	
0	0	0	I_0	Y_0
0	0	1	I_1	
0	1	0	I_2	
0	1	1	I_3	
1	0	0	I_4	Y_1
1	0	1	I_5	
1	1	0	I_6	
1	1	1	I_7	

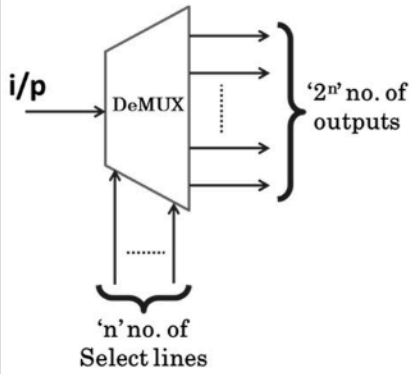


141

141

Demultiplexer (DeMUX)

Logic Symbol & Functionality

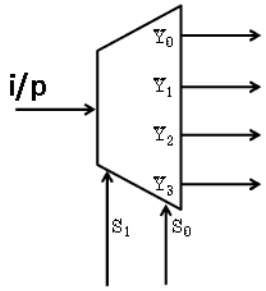


- It's operation is just opposite to that of a Multiplexer.
- Depending on the data on the select lines, the input will be redirected to any one specific output line.

142

142

1:4 DeMUX



Expression for output

$Y_0 = \overline{S_1}\,\overline{S_0}\,I$

$Y_1 = \overline{S_1}\,S_0\,I$

$Y_2 = S_1\,\overline{S_0}\,I$

$Y_3 = S_1 S_0 I$

143

143

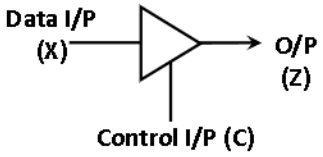
Tristate Buffer

144

144

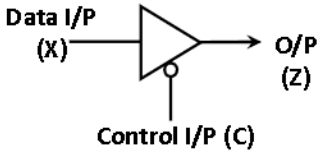
Tri State Buffers

Tristate Buffer with Active HIGH Control



C	X	Carry
0	0	Z
0	1	Z
1	0	0
1	1	1

Tristate Buffer with Active LOW Control

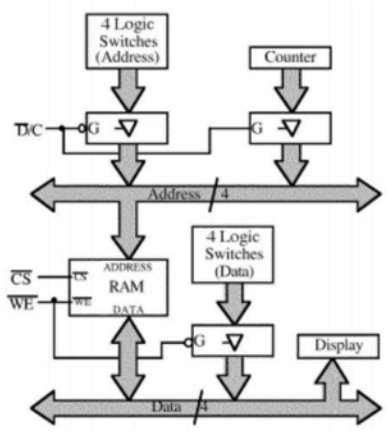


C	X	Carry
0	0	0
0	1	1
1	0	Z
1	1	Z

145

145

Memory Circuit



$\overline{D}$ / C	Mode
0	Switches
1	Counter

$\overline{WE}$	Mode
0	Write
1	Read

146

146